

TP 7 : Apprentissage supervisé et arbre de décision

Compléments de Mathématiques 2 - L3 MIASHS

HAMLIL Mohamed

2026-03-17

Table des matières

1	Préparation des données	2
1.1	Chargement du jeu de données	2
1.2	Vérification des types	2
2	Arbre de décision	3
2.1	Variables explicatives utilisées pour la prédiction	3
3	Arbre de décision avec Holdout	4
3.1	Partage du jeu de données	4
3.2	Gestion des données manquantes	4
3.3	Phase d'apprentissage	4
3.4	Prédiction et évaluation sur le jeu de test	5
3.5	Élagage de l'arbre	7
3.6	Prédiction avec l'arbre élagué	9
4	Arbre de décision avec 10-fold cross-validation	10
4.1	Sans élagage	10
4.2	Avec élagage	13

1 Préparation des données

1.1 Chargement du jeu de données

```
qualitative_vars <- read.csv("../TP2/titanic_pre_processed_qualitative_vars.csv", stringsAsFactors=FALSE)
quantitative_vars <- read.csv("../TP2/titanic_pre_processed_quantitative_vars.csv")
target <- read.csv("../TP2/titanic_pre_processed_target.csv")
target <- as.factor(target$x)
```

Suppression des observations avec valeurs manquantes pour Sex (seulement 2 sur 876, soit < 0.2%) :

```
isMissingSex <- which(is.na(qualitative_vars$Sex) | qualitative_vars$Sex == "")
if (length(isMissingSex) > 0) {
  qualitative_vars <- qualitative_vars[-isMissingSex, ]
  quantitative_vars <- quantitative_vars[-isMissingSex, ]
  target <- target[-isMissingSex]
}
```

1.2 Vérification des types

```
qualitative_vars$Pclass <- as.factor(qualitative_vars$Pclass)
qualitative_vars$Sex <- as.factor(qualitative_vars$Sex)
qualitative_vars$withFam <- as.factor(qualitative_vars$withFam)

str(qualitative_vars)
```

```
'data.frame': 874 obs. of 4 variables:
 $ Age_cat: Factor w/ 4 levels "Adolescent","Adulte",...: 2 2 2 2 2 NA 2 3 2 1 ...
 $ Pclass : Factor w/ 3 levels "1","2","3": 3 1 3 1 3 3 1 3 3 2 ...
 $ Sex : Factor w/ 3 levels "", "female", "male": 3 2 2 2 3 3 3 3 2 2 ...
 $ withFam: Factor w/ 2 levels "0","1": 2 2 1 2 1 1 1 2 2 2 ...
```

```
str(quantitative_vars)
```

```
'data.frame': 874 obs. of 5 variables:
 $ Age : num 22 38 26 35 35 NA 54 2 27 14 ...
```

```
$ SibSp: int  1 1 0 1 0 0 0 3 0 1 ...
$ Parch: int  0 0 0 0 0 0 0 1 2 0 ...
$ Fam  : int  1 1 0 1 0 0 0 4 2 1 ...
$ Fare : num  7.25 71.28 7.92 53.1 8.05 ...
```

```
str(target)
```

```
Factor w/ 2 levels "Décédé","Survivant": 1 2 2 2 1 1 1 1 2 2 ...
```

2 Arbre de décision

2.1 Variables explicatives utilisées pour la prédiction

Encoding et normalisation

Pour construire un arbre de décision, on n'a **pas besoin** de transformer les variables qualitatives (dummy coding ou label encoding) ni de normaliser/standardiser les variables quantitatives. L'arbre travaille avec des règles de division sur chaque variable individuellement, il gère nativement les variables qualitatives et quantitatives sur des échelles différentes.

On exclut Fam, Parch et SibSp puisqu'on utilisera withFam :

```
predictive_vars <- data.frame(
  qualitative_vars[, c("Pclass", "Sex", "withFam")],
  Age = quantitative_vars$Age,
  Fare = quantitative_vars$Fare
)
str(predictive_vars)
```

```
'data.frame':  874 obs. of  5 variables:
 $ Pclass : Factor w/ 3 levels "1","2","3": 3 1 3 1 3 3 1 3 3 2 ...
 $ Sex    : Factor w/ 3 levels "", "female", "male": 3 2 2 2 3 3 3 3 2 2 ...
 $ withFam: Factor w/ 2 levels "0","1": 2 2 1 2 1 1 1 2 2 2 ...
 $ Age    : num  22 38 26 35 35 NA 54 2 27 14 ...
 $ Fare   : num  7.25 71.28 7.92 53.1 8.05 ...
```

3 Arbre de décision avec Holdout

3.1 Partage du jeu de données

```
set.seed(1234)
n <- nrow(predictive_vars)
inTrain <- sample(1:n, size = round(0.8 * n))

predictive_vars_train <- predictive_vars[inTrain, ]
predictive_vars_test  <- predictive_vars[-inTrain, ]
target_train          <- target[inTrain]
target_test           <- target[-inTrain]
```

3.2 Gestion des données manquantes

On utilise la médiane imputation pour **Age**, en calculant la médiane sur le Train uniquement pour éviter le data leakage :

```
library(caret)

Proc <- preProcess(predictive_vars_train, method = "medianImpute")
predictive_vars_train <- predict(Proc, predictive_vars_train)
predictive_vars_test  <- predict(Proc, predictive_vars_test)
```

3.3 Phase d'apprentissage

```
library(rpart)
library(rpart.plot)
```

La ligne ci-dessous entraîne un arbre de décision de classification (`method = "class"`) sur les données d'entraînement. La formule `target_train ~ .` signifie que l'on utilise toutes les variables de `predictive_vars_train` pour prédire `target_train`.

```
classifTree <- rpart(target_train ~ ., data = predictive_vars_train,
                     method = "class")
```

i Critère de division

Par défaut, `rpart` utilise l'**indice de Gini** (impureté de Gini) pour choisir la meilleure règle de division à chaque noeud. On peut changer ce critère en utilisant l'argument `parms = list(split = "information")` pour utiliser l'**entropie** (gain d'information).

Visualisation de l'arbre :

```
rpart.plot(x = classifTree, type = 3, extra = 2, fallen.leaves = TRUE,
           main = "Arbre de prédiction de la survie des passagers au naufrage du Titanic")
```

Arbre de prédiction de la survie des passagers au naufrage du Titanic

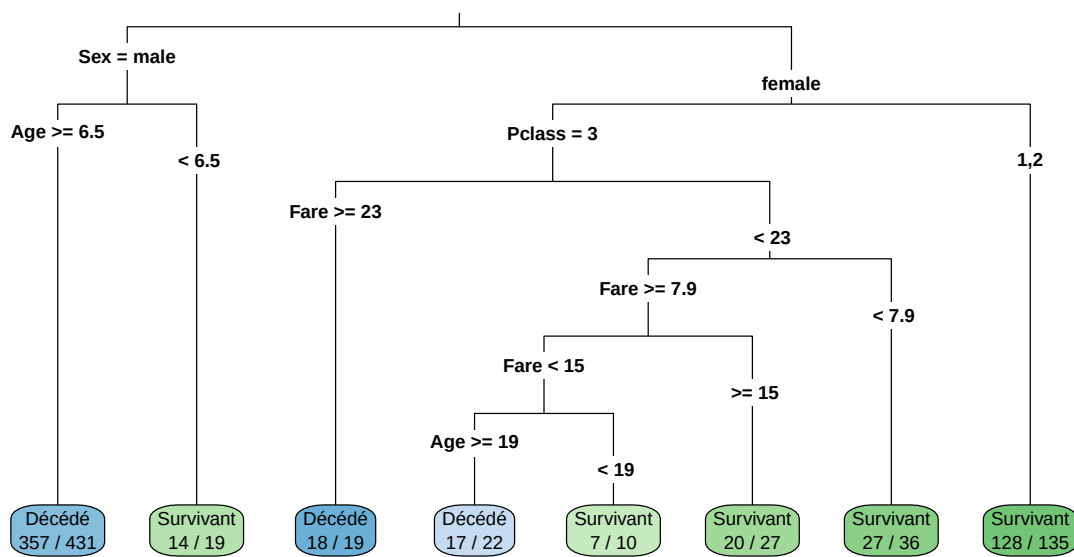


FIGURE 1 : Arbre de décision (sans élagage)

3.4 Prédiction et évaluation sur le jeu de test

```
target_pred <- predict(classifTree, predictive_vars_test, type = "class")
cm <- confusionMatrix(target_pred, target_test, positive = "Survivant")
cm
```

Confusion Matrix and Statistics

	Reference	
Prediction	Décédé	Survivant
Décédé	99	22
Survivant	12	42

Accuracy : 0.8057
 95% CI : (0.7392, 0.8615)
 No Information Rate : 0.6343
 P-Value [Acc > NIR] : 6.27e-07

 Kappa : 0.5669

 McNemar's Test P-Value : 0.1227

 Sensitivity : 0.6562
 Specificity : 0.8919
 Pos Pred Value : 0.7778
 Neg Pred Value : 0.8182
 Prevalence : 0.3657
 Detection Rate : 0.2400
 Detection Prevalence : 0.3086
 Balanced Accuracy : 0.7741

 'Positive' Class : Survivant

```
cat("Accuracy :", round(cm$overall["Accuracy"], 4), "\n")
```

Accuracy : 0.8057

```
cat("Recall :", round(cm$byClass["Sensitivity"], 4), "\n")
```

Recall : 0.6562

```
cat("Precision :", round(cm$byClass["Pos Pred Value"], 4), "\n")
```

Precision : 0.7778

3.5 Élagage de l'arbre

La fonction `plotcp` affiche l'erreur de validation croisée (relative cross-validation error) en fonction du paramètre de complexité `cp`. Cela permet de choisir la bonne taille d'arbre pour éviter le surapprentissage.

```
plotcp(classifTree, upper = "splits")
```

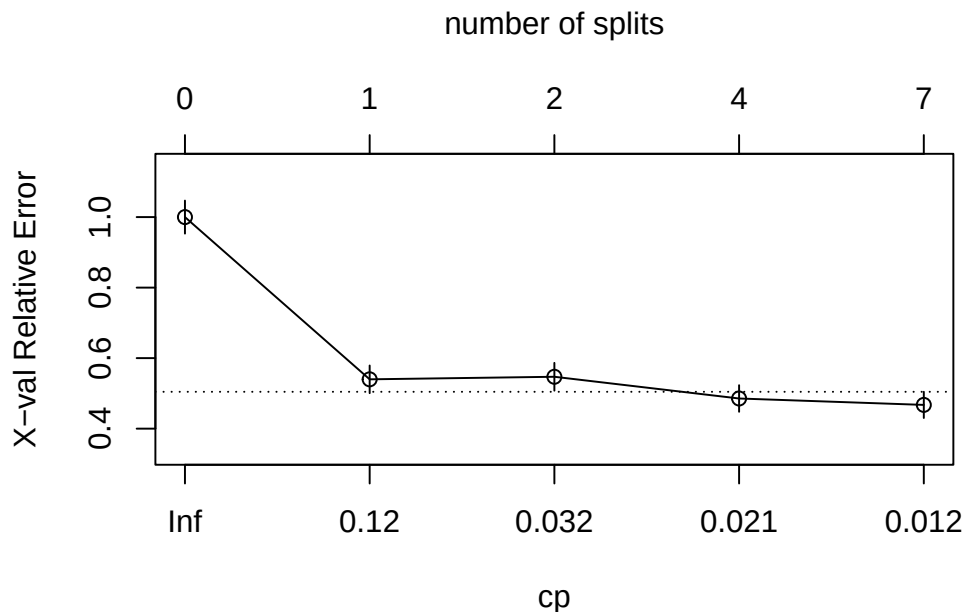


FIGURE 2 : Erreur de CV relative en fonction du paramètre de complexité

💡 Choix du cp optimal

Selon la documentation, un bon choix est la valeur de `cp` correspondant à l'arbre le plus petit dont l'erreur de CV relative est **à moins d'un écart-type** du minimum (règle du "1-SE"). La ligne horizontale en pointillés sur le graphique correspond à $\min(\text{xerror}) + \text{xstd}$ au minimum.

```
classifTree$cptable
```

	CP	nsplit	rel error	xerror	xstd
1	0.46014493	0	1.0000000	1.0000000	0.04682492
2	0.03260870	1	0.5398551	0.5398551	0.03923077
3	0.03079710	2	0.5072464	0.5471014	0.03942131

4	0.01449275	4	0.4456522	0.4855072	0.03770761
5	0.01000000	7	0.4021739	0.4673913	0.03716076

```
xerror      <- classifTree$cptable[, "xerror"]
xstd        <- classifTree$cptable[, "xstd"]
cp          <- classifTree$cptable[, "CP"]
min.xerror  <- which.min(xerror)

# Règle du 1-SE : plus petit arbre dont xerror < min(xerror) + xstd au minimum
threshold <- xerror[min.xerror] + xstd[min.xerror]
bestcp_idx <- min(which(xerror <= threshold))
bestcp     <- cp[bestcp_idx]

cat("Meilleur cp (règle 1-SE) :", bestcp, "\n")
```

Meilleur cp (règle 1-SE) : 0.01449275

```
cat("Nombre de splits associé :", classifTree$cptable[bestcp_idx, "nsplit"], "\n")
```

Nombre de splits associé : 4

Élagage de l'arbre avec la valeur optimale du cp :

```
prunedTree <- prune(classifTree, cp = bestcp)
```

```
rpart.plot(prunedTree, type = 3, extra = 2, fallen.leaves = TRUE,
           main = "Arbre élagué")
```


Arbre élagué

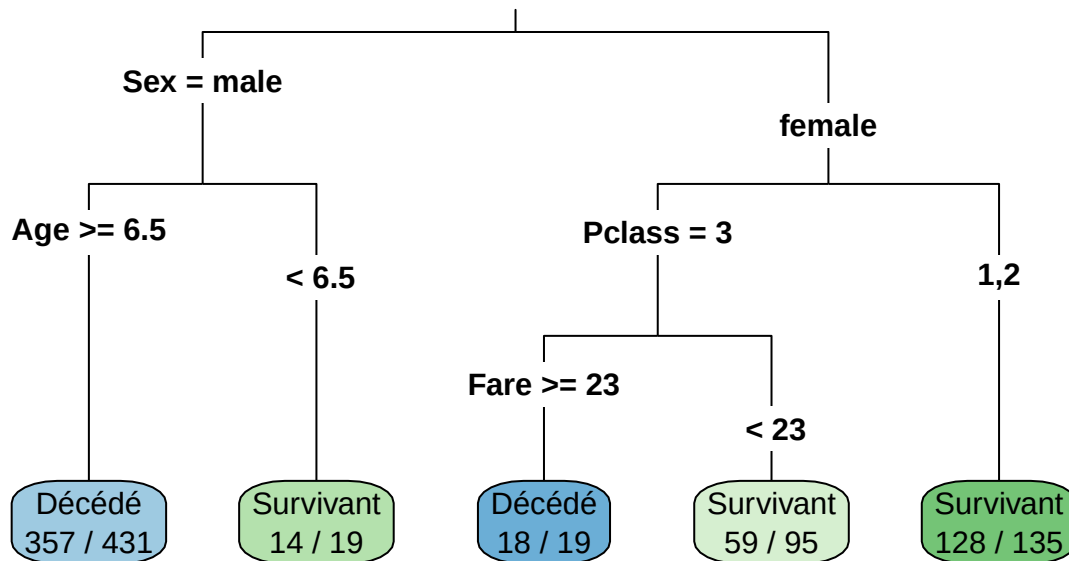


FIGURE 3 : Arbre de décision élagué

3.6 Prédiction avec l'arbre élagué

```
target_pred_pruned <- predict(prunedTree, predictive_vars_test, type = "class")
cm_pruned <- confusionMatrix(target_pred_pruned, target_test, positive = "Survivant")
cm_pruned
```

Confusion Matrix and Statistics

	Reference	
Prediction	Décédé	Survivant
Décédé	94	19
Survivant	17	45

Accuracy : 0.7943
 95% CI : (0.7268, 0.8516)
 No Information Rate : 0.6343
 P-Value [Acc > NIR] : 3.434e-06

Kappa : 0.5536

McNemar's Test P-Value : 0.8676

Sensitivity : 0.7031
 Specificity : 0.8468
 Pos Pred Value : 0.7258
 Neg Pred Value : 0.8319
 Prevalence : 0.3657
 Detection Rate : 0.2571
 Detection Prevalence : 0.3543
 Balanced Accuracy : 0.7750

'Positive' Class : Survivant

```
cat("Accuracy (élagué) :", round(cm_pruned$overall["Accuracy"], 4), "\n")
```

Accuracy (élagué) : 0.7943

```
cat("Recall (élagué) :", round(cm_pruned$byClass["Sensitivity"], 4), "\n")
```

Recall (élagué) : 0.7031

```
cat("Precision (élagué) :", round(cm_pruned$byClass["Pos Pred Value"], 4), "\n")
```

Precision (élagué) : 0.7258

i Comparaison

L'arbre élagué est plus simple (moins de noeuds) tout en conservant des performances comparables, voire identiques, à l'arbre complet. C'est le principe du rasoir d'Occam : on préfère le modèle le plus simple qui explique aussi bien les données. L'élagage réduit le risque de surapprentissage.

4 Arbre de décision avec 10-fold cross-validation

4.1 Sans élagage

```

set.seed(1234)
nfolds <- 10
folds <- sample(1:nfolds, n, replace = TRUE)

accuracy_fold <- numeric(nfolds)
recall_fold <- numeric(nfolds)
precision_fold <- numeric(nfolds)

for (k in 1:nfolds) {
  inFold <- which(folds == k)

  predictive_vars_test <- predictive_vars[inFold, ]
  target_test <- target[inFold]
  predictive_vars_train <- predictive_vars[-inFold, ]
  target_train <- target[-inFold]

  # Median imputation (sans data leakage)
  Proc <- preProcess(predictive_vars_train, method = "medianImpute")
  predictive_vars_train <- predict(Proc, predictive_vars_train)
  predictive_vars_test <- predict(Proc, predictive_vars_test)

  # Apprentissage
  classifTree <- rpart(target_train ~ ., data = predictive_vars_train,
                      method = "class")

  # Prédiction
  target_pred <- predict(classifTree, predictive_vars_test, type = "class")

  # Métriques
  cm <- confusionMatrix(target_pred, target_test, positive = "Survivant")

  accuracy_fold[k] <- cm$overall["Accuracy"]
  recall_fold[k] <- cm$byClass["Sensitivity"]
  precision_fold[k] <- cm$byClass["Pos Pred Value"]
}

names(accuracy_fold) <- paste0("fold_", 1:nfolds)
names(recall_fold) <- paste0("fold_", 1:nfolds)
names(precision_fold) <- paste0("fold_", 1:nfolds)

```

`accuracy_fold`

fold_1	fold_2	fold_3	fold_4	fold_5	fold_6	fold_7	fold_8
0.8111111	0.7000000	0.8333333	0.7792208	0.7674419	0.7787611	0.8045977	0.8426966
fold_9	fold_10						
0.8289474	0.8750000						

`recall_fold`

fold_1	fold_2	fold_3	fold_4	fold_5	fold_6	fold_7	fold_8
0.7187500	0.3750000	0.6250000	0.6571429	0.6315789	0.6481481	0.5757576	0.6875000
fold_9	fold_10						
0.7931034	0.7741935						

`precision_fold`

fold_1	fold_2	fold_3	fold_4	fold_5	fold_6	fold_7	fold_8
0.7419355	0.5000000	0.8333333	0.8214286	0.8000000	0.8536585	0.8636364	0.8461538
fold_9	fold_10						
0.7666667	0.8888889						

```
cat("10-fold CV Mean Accuracy  :", round(mean(accuracy_fold), 4), "\n")
```

```
10-fold CV Mean Accuracy  : 0.8021
```

```
cat("10-fold CV Mean Recall    :", round(mean(recall_fold), 4), "\n")
```

```
10-fold CV Mean Recall      : 0.6486
```

```
cat("10-fold CV Mean Precision :", round(mean(precision_fold, na.rm = TRUE), 4), "\n")
```

```
10-fold CV Mean Precision : 0.7916
```

💡 Nested cross-validation

On fait ici de la **nested cross-validation** : la boucle externe (nos 10 folds) sert à évaluer le classifieur, tandis que la boucle interne (réalisée automatiquement par `rpart`) utilise la cross-validation pour choisir la meilleure valeur du paramètre de complexité `cp`.

4.2 Avec élagage

```
accuracy_fold_p <- numeric(nfolds)
recall_fold_p   <- numeric(nfolds)
precision_fold_p <- numeric(nfolds)

for (k in 1:nfolds) {
  inFold <- which(folds == k)

  predictive_vars_test <- predictive_vars[inFold, ]
  target_test         <- target[inFold]
  predictive_vars_train <- predictive_vars[-inFold, ]
  target_train        <- target[-inFold]

  # Median imputation (sans data leakage)
  Proc <- preProcess(predictive_vars_train, method = "medianImpute")
  predictive_vars_train <- predict(Proc, predictive_vars_train)
  predictive_vars_test  <- predict(Proc, predictive_vars_test)

  # Apprentissage
  classifTree <- rpart(target_train ~ ., data = predictive_vars_train,
                       method = "class")

  # Élagage (règle du 1-SE)
  xerror      <- classifTree$cptable[, "xerror"]
  xstd        <- classifTree$cptable[, "xstd"]
  cp_vals     <- classifTree$cptable[, "CP"]
  min_idx     <- which.min(xerror)
  threshold   <- xerror[min_idx] + xstd[min_idx]
  best_idx    <- min(which(xerror <= threshold))
  best_cp     <- cp_vals[best_idx]

  prunedTree  <- prune(classifTree, cp = best_cp)
```

```
# Prédiction avec l'arbre élagué
target_pred <- predict(prunedTree, predictive_vars_test, type = "class")

# Métriques
cm <- confusionMatrix(target_pred, target_test, positive = "Survivant")

accuracy_fold_p[k] <- cm$overall["Accuracy"]
recall_fold_p[k] <- cm$byClass["Sensitivity"]
precision_fold_p[k] <- cm$byClass["Pos Pred Value"]
}

names(accuracy_fold_p) <- paste0("fold_", 1:nfolds)
names(recall_fold_p) <- paste0("fold_", 1:nfolds)
names(precision_fold_p) <- paste0("fold_", 1:nfolds)
```

```
accuracy_fold_p
```

```
      fold_1    fold_2    fold_3    fold_4    fold_5    fold_6    fold_7    fold_8
0.7666667 0.7000000 0.8020833 0.7792208 0.7674419 0.7876106 0.8045977 0.8202247
      fold_9    fold_10
0.7894737 0.8500000
```

```
recall_fold_p
```

```
      fold_1    fold_2    fold_3    fold_4    fold_5    fold_6    fold_7    fold_8
0.7500000 0.4166667 0.6250000 0.6857143 0.6052632 0.6296296 0.7575758 0.7187500
      fold_9    fold_10
0.7586207 0.8064516
```

```
precision_fold_p
```

```
      fold_1    fold_2    fold_3    fold_4    fold_5    fold_6    fold_7    fold_8
0.6486486 0.5000000 0.7407407 0.8000000 0.8214286 0.8947368 0.7352941 0.7666667
      fold_9    fold_10
0.7096774 0.8064516
```

```
cat("10-fold CV Mean Accuracy (élagué) :", round(mean(accuracy_fold_p), 4), "\n")
```

```
10-fold CV Mean Accuracy (élagué) : 0.7867
```

```
cat("10-fold CV Mean Recall      (élagué) :", round(mean(recall_fold_p), 4), "\n")
```

```
10-fold CV Mean Recall      (élagué) : 0.6754
```

```
cat("10-fold CV Mean Precision (élagué) :", round(mean(precision_fold_p, na.rm = TRUE), 4),
```

```
10-fold CV Mean Precision (élagué) : 0.7424
```

i Comparaison

Les performances avec et sans élagage sont généralement très proches, ce qui confirme que l'élagage simplifie l'arbre sans dégrader significativement les performances. L'arbre élagué est préférable car il est plus interprétable et moins susceptible de surapprentissage.